Federated Class-Incremental Learning With Dynamic Feature Extractor Fusion

Yanyan Lu, Lei Yang, *Member, IEEE*, Hao-Rui Chen, Jiannong Cao, *Fellow, IEEE*, Wanyu Lin, *Member, IEEE*, and Saiqin Long

***Abstract*—Federated class-incremental learning (FCIL) allows multiple clients in a distributed environment to learn models collaboratively from evolving data streams, where new classes arrive continually at each client. Some existing works in FCIL combine traditional federated learning methods with class-incremental methods. However, the global model affected by data heterogeneity can aggravate local forgetting through the direct combination of traditional methods. To tackle this issue, we propose FCIDF, a novel Federated Class-Incremental learning approach based on *Dynamic feature extractor Fusion*. FCIDF learns personalized and incremental models for each client by introducing personalized fusion rates to integrate global knowledge into local features. Leveraging *meta-learning* during each incremental round, FCIDF ensures involvement of both old and new task knowledge in personalized training. Besides, we further propose a new Storing strategy based on Accumulated Global Feature Means (AGFMS), which helps the model review unbiased old knowledge and compensates for local forgetting. Experiment results show that FCIDF outperforms the baseline methods in both accuracy and forgetting on most settings, and AGFMS improves the performance of FCIDF on most evaluated scales.**

***Index Terms*—Federated learning, class-Incremental learning, feature extractor, exemplar storing.**

I. Introduction

With the rapid development of Artificial Intelligence (AI), deep learning is widely deployed on edge devices for various applications, such as traffic surveillance [1], smart home [2] and defect detection [3]. The edge devices generate data continually and may not be willing to share the data due to privacy concerns. Federated learning [4], [5] is a distributed learning framework, which allows clients to collaborate with each other to learn a global model without exchanging their raw data. Learning a shared model among clients often suffers from data heterogeneity, where feature distributions or label distributions might vary tremendously among clients. It has been proved that clients' data heterogeneity, or called non-Independent and Identical Distribution (non-IID), degrades the performance of the shared model [6], which poses a great challenge in federated learning. To solve the non-IID problem, recent works propose several solutions such as data augmentation [7], [8], client selection and clustering [9], [10], [11], and personalized learning [12], [13], [14].

Most existing works on federated learning focus on the scenario where the clients' local data is fixed. However, real-world devices often receive a dynamic stream of data. This further increases the difficulty of federated learning because data heterogeneity might happen both spatially and temporally. To tackle this issue, some researchers [15], [16], [17] propose integrating federated learning with incremental learning. Incremental learning considers the scenario where old data is no longer applicable and only new data is available for training. Among others, class-incremental learning focuses on the case that new classes appear continually, and a single model should be trained incrementally to classify all the observed classes. Traditional solutions for class-incremental learning like regularization-based methods [18] and replay-based methods [19], [20], [21] tackled the catastrophic forgetting in standalone computing. However, directly applying these solutions in federated learning results in poor model accuracy and convergence performance. This is because, in a distributed environment, the catastrophic forgetting could be aggravated since local training typically relies on a globally aggregated model infected by non-IID data. Besides, the clients' models have various output layers due to different new classes arriving at each client. Therefore, the output layers cannot be directly aggregated into a global model.

In this paper, we focus on the above challenges in Federated Class-Incremental Learning (FCIL). At an incremental round, each client receives a new task with classes unobserved before, but might have been observed by other clients. We aim to utilize other clients' knowledge to learn a personalized model that performs well in local observed classes for each client. The main challenge is to solve spatial non-IID data and temporal catastrophic forgetting simultaneously. More specifically, clients within a network might not follow the same label distribution, i.e., non-IID. Meanwhile, each client continually

Manuscript received 7 April 2024; revised 17 June 2024; accepted 23 June 2024. Date of publication 26 June 2024; date of current version 5 November 2024. This work was supported in part by the NSFC and Hong Kong RGC Collaborative Research Scheme under Grant 62321166652, in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2022A1515010374, in part by Hong Kong RGC Theme-based Research Scheme (TRS) under Grant T43-513/23-N, and in part by the National Natural Science Foundation of China under Grant U23B2027. Recommended for acceptance by D. T. Hoang. *(Corresponding author: Lei Yang.)*

Yanyan Lu, Lei Yang, and Hao-Rui Chen are with the School of Software Engineering, South China University of Technology, Guangzhou 510006, China (e-mail: seyylu@mail.scut.edu.cn; sely@scut.edu.cn; sechenhr@mail.scut.edu.cn).

Jiannong Cao and Wanyu Lin are with the Department of Computing, Hung Hom, Hong Kong (e-mail: csjcao@comp.polyu.edu.hk; wanylin@comp.polyu.edu.hk).

Saiqin Long is with the College of Information Science and Technology, Jinan University, Guangzhou 510632, China (e-mail: saiqinlong@jnu.edu.cn).

Digital Object Identifier 10.1109/TMC.2024.3419096

receives new data with new classes and cannot save all old data due to limited memory. Recent works on federated class-incremental learning [15], [16], [22] aim to learn a global shared model by distillation-based methods, and other works [23], [24], [25] learn a generator to memorize knowledge from past tasks. These methods can relieve catastrophic forgetting in class-incremental learning. However, a single model can gather limited global information for all clients, and generative models make the computational and communication costs grow rapidly for clients. Besides, the global model affected by non-IID data can aggravate local forgetting through the direct combination of federated learning methods and class-incremental learning methods.

To tackle the above problems, we propose a novel federated class-incremental learning approach with dynamic feature extractor fusion, where each client dynamically utilizes knowledge from other clients to learn its own personalized model that can classify its local observed classes. Specifically, by fusing the local feature extractor (local FE) with the global feature extractor (global FE) via a learnable rate vector, each client learns a personalized model that fits its local data from the global knowledge and avoids performance decrease caused by non-IID data. To address the forgetting problem caused by imbalanced data, we leverage meta-learning to train the local model and the fusion rate, where the meta tasks are derived from the new data and stored exemplars. This strategy involves the knowledge of both new and old tasks in personalized training, so that the non-IID data and catastrophic forgetting problems can be solved simultaneously. In addition, we modify the local storing method of exemplars and propose to Store based on Accumulated Global Feature Means (AGFMS). This strategy relieves the biases among clients' old knowledge of the same class, so that the noise within the global feature extractor can be mitigated and the performance of the local model can be improved. The experiment results show that FCIDF obtains up to 7.69% improvement in accuracy and 27.30% in forgetting on CIFAR-100 and ILSVRC2012 datasets, compared to multiple baseline methods. We also conduct extensive experiments on AGFMS and observe that AGFMS improves up to 3.88% in accuracy and 3.51% in forgetting compared to the original local storing method. The main contributions of this paper can be summarized as follows:

- We focus on the challenging federated class-incremental learning settings and propose a new approach to overcoming data heterogeneity spatially and temporally.
- Our approach uses dynamic feature extractor fusion to learn a personalized model for each client, and leverage meta-learning to update the parameters so that both old and new knowledge can be involved in the personalized training.
- We design a novel Storing method of exemplars based on Accumulated Global Feature Means (AGFMS), which stores exemplars that represent unbiased global old knowledge of each class and improves the model performance.
- We have done extensive experiments to show that our method outperforms most baseline methods in both accuracy and forgetting.

II. Related Work

A. Federated Learning

Federated learning [4], [5] is a popular framework to learn on distributed and private data, which utilizes a central server to integrate clients' local models into a globally shared model using weighted aggregation, namely FedAvg. Federated learning involves a wide range of problems, like client selection [9], [26], adversarial attacks [27] and data heterogeneity [6], [28], [29]. Among others, data heterogeneity is a basic problem since the data in real world is often non-independently and identically distributed (non-IID) [6]. FedProx [28] allows each client to perform a various amount of work locally according to their available resources, while using a proximal term to keep their local updates stable. SCAFFOLD [29] aims to reduce the variance among the clients caused by non-IID data. Personalized federated learning [12], [13], [14] aims to obtain a personalized model for each client, which can better adapt to the client's local data and thus relieve the effects of non-IID data.

B. Class-Incremental Learning

Class-incremental learning [19], [20], [21] focuses on the scenario that the continuous data is from the same classification task, and new classes arrive over time. The samples should be classified within a unified classifier that can distinguish all observed classes.

Existing methods for class-incremental learning can be categorized into two categories: regularization and replay. Regularization-based methods [18], [30] use a regularization term in the loss function to prevent the model from forgetting old knowledge. Knowledge Distillation (KD) [31] loss is widely adopted for regularization, using the outputs of the old model as soft labels to transfer old knowledge to the new model. Replay-based methods [19], [20], [21] store a certain amount of old data and replay them in future learning, so that old knowledge can be reviewed when learning new knowledge. Recently, meta-learning [32], [33] is used to overcome catastrophic forgetting in class-incremental learning. Meta-learning extracts the common knowledge of multiple tasks and adapts to new tasks quickly, so incremental-meta learning [34], [35] can learn the shared knowledge of new tasks and stored old tasks. However, combining the above methods with federated learning poses new challenges to class-incremental learning that catastrophic forgetting may be aggravated due to data heterogeneity among clients.

C. Federated Incremental Learning

Due to the distributed and streaming features of real-world data, federated incremental learning is proposed to deal with this challenging scenario. DCIGAN [23] aims to train a global generator for each observed class that can generate data close to global distribution under the distributed class-incremental learning framework, so the information of all the observed data can be stored. FedWeIT [17] proposes a federated continual learning framework, where the model is decomposed into a globally shared model and multiple task-specific models, and

all task-specific models are sent to each client for knowledge transfer. The memory cost of the above two methods increases significantly on each client since the number of models grows as new data arrives. FedET [36] optimizes enhancers for a pre-trained transformer on different tasks, which reduces computational costs as enhancers have fewer parameters. Yet, the model accuracy is limited by the pre-trained transformer, whose parameters are frozen. GLFC [16] and LGA [22] learn a global class-incremental model to solve global and local forgetting simultaneously, which utilizes the gradients of samples to help the server choose the best old model for distillation. However, a single model can be interfered by the non-IID environment more easily. MFCL [24] and TARGET [25] train a generator in a data-free manner to generate synthesized samples of various categories, which keeps the privacy of client datasets but significantly increases the communication cost and computational cost of clients.

In this paper, we propose a novel federated class-incremental learning approach, namely FCIDF, where each client trains a personalized and incremental model to classify its local data and only the output layer of the model is expanded when new classes come.

III. Problem Statement

In this section, we would like to introduce our settings for Federated Class-Incremental Learning (FCIL).

There are N clients in the network, and each client will receive T incremental tasks. During an incremental round t, $t \in \{1, 2, \ldots, T\}$, each client receives a task $\mathcal{T}_n^t = \{\mathbf{x}_{in}^t, y_{in}^t\}_{i=1}^{\mathcal{N}_n^t}$, where $n \in \{1, 2, \ldots, N\}$ and $\mathcal{N}_n^t$ denotes the number of samples in $\mathcal{T}_n^t$. $\mathcal{T}_n^t$ contains data of C classes, which are new to client n but may have been observed by other clients. Besides, each client maintains a fixed size of memory to store exemplars from each task. We denote the stored exemplars of client n as $\mathcal{M}_n^t$ which contains M old samples. At incremental round t, the memory contains samples of the classes observed at round 1 to $t-1$, and each class has $\frac{M}{C(t-1)}$ exemplars.

After the clients receive their new tasks, they utilize the global model $\mathbf{\Theta}_{glob}^t$ aggregated at the last round to construct the local model $\mathbf{\Theta}_n^t$, and train $\mathbf{\Theta}_n^t$ with the new task $\mathcal{T}_n^t$ and stored exemplars $\mathcal{M}_n^t$. We divide $\mathbf{\Theta}_n^t$ into two parts, feature extractor $\boldsymbol{\theta}_n^t$ and classifier $\boldsymbol{\phi}_n^t$, where the classifier is the output layer of the model, and the rest layers form the feature extractor. Since new classes appear over time, the classifier's structure should be changed to classify new classes. At incremental round t, we add C neurons to the classifier, so the classifier can be learned to classify $C \times t$ classes.This leads to a new problem that classifiers are semantically heterogeneous among clients, since each client may receive different new classes and the clients train their models under various label distributions. The aggregation of heterogeneous classifiers may disturb the local classification bounds of the clients and cause privacy concerns. We also conduct extensive experiments to prove that aggregating heterogeneous classifiers is unnecessary. For the above reasons, we keep the classifier locally and only upload the feature extractor for aggregation.

TABLE I
Notation

Notation	Definition
N	The number of clients, indexed by n
T	The number of incremental rounds, indexed by t
C	The number of classes within an incremental task
E	The number of epochs of local training
R	The number of meta-rounds in meta-training process
L	The number of layers in the feature extractor
$\mathcal{T}_n^t$	The t-th incremental task received by client n
$\mathcal{M}_n^t$	The stored exemplars on client n at incremental round t
$\mathbf{\Theta}$	The model consisting of feature extractor $\boldsymbol{\theta}$ and classifier $\boldsymbol{\phi}$
$\boldsymbol{\alpha}$	The fusion rate of global and local feature extractors

We summarize the notations of our problem in Table I. The main problem of this paper is to utilize the knowledge from the global model to learn a personalized and incremental model for each client without forgetting the old knowledge.

IV. Proposed Method

In this section, we introduce our Federated Class-Incremental learning method with Dynamic feature extractor Fusion (FCIDF).

A. Dynamic Feature Extractor Fusion

Traditional federated learning methods aggregate the local models into a global model and send it to the clients. The clients replace their local models with the global model for further training. However, the replacing strategy can aggravate local forgetting under the federated class-incremental learning settings. Due to the non-IID data across clients, the global model aggregated by FedAvg may contain some noisy information for each client. Besides, the knowledge from different clients and different incremental tasks is too large to be remembered by a single model. Thus, if the local model is directly replaced by the global model, the client's previous knowledge can be infected by unrelated knowledge and critically forgotten. For this reason, we attempt to keep both the local model and the global model locally, and propose an effective way, *dynamic feature extractor fusion* (FCIDF), to relieve catastrophic forgetting and learn new knowledge simultaneously.

The process of FCIDF is shown in Fig. 1. Due to the heterogeneity of clients' classifiers, the only information exchanged between the server and clients is the parameters of the feature extractor. At the beginning of an incremental round t, each client receives the global feature extractor (global FE) aggregated by the server at round $t-1$. The global FE is then saved locally together with the local feature extractor (local FE). Our goal is to leverage the global knowledge inside the global FE without covering the local knowledge. To reach this goal, we fuse the global FE and the local FE via a rate vector $\boldsymbol{\alpha}_n^t$. This vector maintains a fusion weight for each layer of the feature extractor, so it can be denoted as $\{a_l\}_{l=1}^{L}$, where L is the number of layers in the feature extractor. The fused feature extractor is defined as

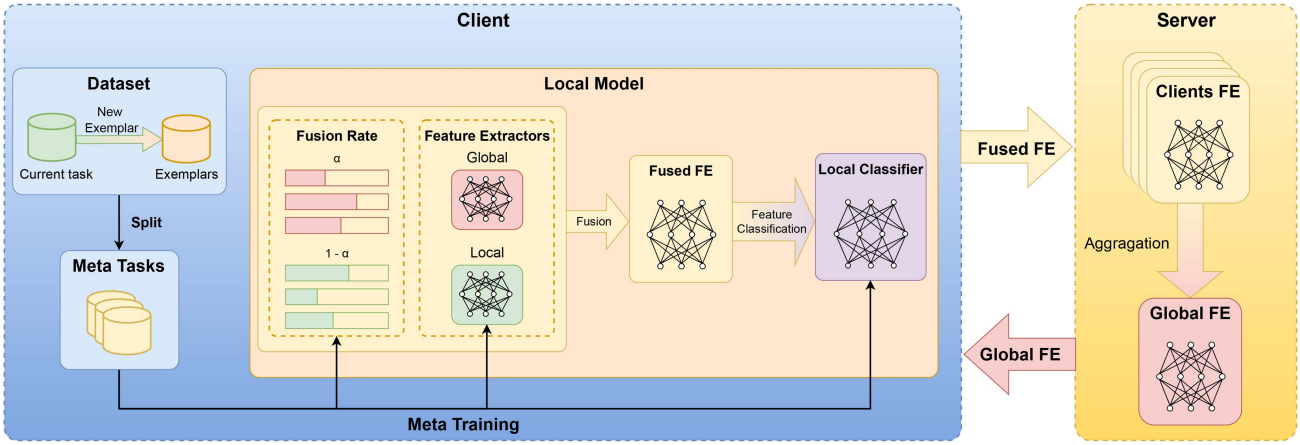

Fig. 1. Overview of FCIDF process. It can be divided into the following steps: ① The server sends global Feature Extractor (FE) to each client; ② The client initializes the fusion rate and the local model; ③ The client initializes the training data; ④ The client performs meta-learning on the current data; ⑤ The client stores exemplars of each observed class and sends the fused local FE to the server; ⑥ The server aggregates the local FEs and obtains a new global FE.

follows:

$$\boldsymbol{\theta}_n^t = \boldsymbol{\alpha}_n^t \times \boldsymbol{\theta}_{glob}^t + (1 - \boldsymbol{\alpha}_n^t) \times \boldsymbol{\theta}_{n,loc}^t, \tag{1}$$

where $\boldsymbol{\theta}_{glob}^t$ is the global feature extractor, and $\boldsymbol{\theta}_{n,loc}^t$ is the local feature extractor obtained from round $t-1$. The fused feature extractor is used to extract samples' features, and these features are passed to the local classifier for final classification.

Before each incremental round's training, $\boldsymbol{\alpha}_n^t$ is randomly initialized and the training data formed by the stored exemplars and the new task is divided into several batches. We adapt the meta-learning process used in iTAML [35] to our training, since we require the model to learn the shared knowledge of both the old tasks and the new tasks. We perform the meta-learning process for E epochs in each incremental round. For each training batch, we split the data into several meta tasks based on their appearing time, and the base model is initialized by the current model $\boldsymbol{\theta}_n^t$. Then, each meta task $\mathcal{T}_k$ trains a copy of the base model $\boldsymbol{\Theta}_{base}$ for R rounds. During training, we keep the global FE $\boldsymbol{\theta}_{glob}^t$ fixed and update the rate vector $\boldsymbol{\alpha}_k$, the local FE $\boldsymbol{\theta}_{k,loc}$ and the local classifier $\boldsymbol{\phi}_k$ to minimize the following loss function:

$$\mathcal{L}(\boldsymbol{\alpha}_k, \boldsymbol{\theta}_{k,loc}, \boldsymbol{\phi}_k) = \sum_{i=1}^{|\mathcal{T}_k|} l(g(\boldsymbol{\phi}_k; f(\boldsymbol{\alpha}_k, \boldsymbol{\theta}_{glob}^t, \boldsymbol{\theta}_{k,loc}; \mathbf{x}_i)); y_i), \tag{2}$$

where l is the cross-entropy loss, f is the mapping function of the feature extractor, g is the classification function. After training, each meta task obtains a task-specific meta model $\boldsymbol{\Theta}_k$. Note that the whole classifier is updated for each meta task since we attempt to train a unified classifier that can distinguish all observed classes, which is different from iTAML. Finally, the meta models are averaged and combined with the base model to obtain a new model:

$$\boldsymbol{\Theta}_{new} = \eta \cdot \frac{1}{t} \sum_{k=1}^{t} \boldsymbol{\Theta}_k + (1 - \eta) \cdot \boldsymbol{\Theta}_{base}, \tag{3}$$

where $\eta = \exp(-\beta \frac{t}{T})$. The new model is used as the base model for the next batch.

When the meta-learning process finishes, the fused feature extractor is transformed into a single feature extractor, which will be used as local feature extractor. Besides, we update clients' stored exemplars locally by selecting the most representative samples of each class following iCaRL [19]. Finally, the server collects the fused feature extractors from all clients and aggregates them via weighted average:

$$\boldsymbol{\theta}_{glob}^{t+1} = \sum_{n=1}^{N} \frac{\mathcal{N}_n^t}{\mathcal{N}^t} \cdot \boldsymbol{\theta}_n^t, \tag{4}$$

where $\mathcal{N}^t$ is the number of all clients' training samples.

Algorithm 1 illustrates the whole process of FCIDF. Through the learning of the fusion rate, the client can dynamically adjust the utilization of the global knowledge based on its data distribution, such that only the information relative to this client is absorbed. Besides, meta-learning extracts the shared knowledge from both new tasks and old tasks to perform personalized training, relieving catastrophic forgetting brought by federated learning. Though the old tasks have fewer samples than the new tasks, the meta models are trained independently for each task so that the new model would not bias towards the new tasks.

B. Exemplar Storing in Federated Class-Incremental Learning

1) Motivation of Storing Exemplars Based on Global Information: In FCIDF, we already share the distributed information through a global feature extractor, which includes shared knowledge among all global classes. Further, we can also generate global class-related information to share knowledge of a specific class, so that the clients owning the same class can learn the class' knowledge collaboratively and compensate for the forgotten knowledge.

We observe from FCIDF that storing exemplars has a strong relationship with class-specific knowledge. The sample selection of each class depends on the feature mean where the features are extracted by the current feature extractor. In other words, the

Algorithm 1: FCIDF Process of Incremental Round t.

Input: Global feature extractor $\boldsymbol{\theta}_{glob}^{t}$
Output: New global feature extractor $\boldsymbol{\theta}_{glob}^{t+1}$
1: **if** $t = 1$ **then**
2: Initialize global feature extractor $\boldsymbol{\theta}_{glob}^{1}$.
3: **end if**
4: **for** each client n in $1, 2, \ldots, N$ parallel **do**
5: Send $\boldsymbol{\theta}_{glob}^{t}$ to client n.
6: $\boldsymbol{\theta}_{n}^{t} \longleftarrow \mathbf{ClientUpdate}(n)$.
7: **end for**
8: Aggregate local feature extractors to obtain $\boldsymbol{\theta}_{glob}^{t+1}$ using (4).

Algorithm 2: $\mathbf{ClientUpdate}(n)$.

Input: Stored exemplars $\mathcal{M}_{n}^{t}$, new task $\mathcal{T}_{n}^{t}$, local model $\boldsymbol{\Theta}_{n}^{t}$
Output: New local model $\boldsymbol{\Theta}_{n}^{t+1}$
1: Receive a new task $\mathcal{T}_{n}^{t}$ and initialize the current data $\mathcal{D}_{n}^{t} = \{\mathcal{T}_{n}^{t}, \mathcal{M}_{n}^{t}\}$.
2: Randomly initialize the fusion rate $\boldsymbol{\alpha}_{n}^{t}$.
3: Receive $\boldsymbol{\theta}_{glob}^{t}$ from the server and initialize $\boldsymbol{\theta}_{n}^{t}$ using (1).
4: **for** each epoch e in $1, 2, \ldots, E$ **do**
5: **for** each training batch from $\mathcal{D}_{n}^{t}$ **do**
6: $\boldsymbol{\Theta}_{base} \longleftarrow \boldsymbol{\Theta}_{n}^{t}$.
7: **for** k in $1, 2, \ldots, t$ **do**
8: Extract samples of the classes appearing at incremental round k from the training batch and form a meta task $\mathcal{T}_{k}$.
9: $\boldsymbol{\Theta}_{k} \longleftarrow \boldsymbol{\Theta}_{base}$.
10: **for** r in $1, 2, \ldots, R$ **do**
11: Fix $\boldsymbol{\theta}_{glob}$ and update $\boldsymbol{\alpha}_{k}$, $\boldsymbol{\theta}_{k,loc}$ and $\boldsymbol{\phi}_{k}$ by minimizing (2) with $\mathcal{T}_{k}$.
12: **end for**
13: **end for**
14: Combine the base model with the meta models using (3) and update $\boldsymbol{\Theta}_{n}^{t} \longleftarrow \boldsymbol{\Theta}_{new}$.
15: **end for**
16: **end for**
17: $\boldsymbol{\theta}_{n,loc}^{t+1} \longleftarrow \boldsymbol{\theta}_{n}^{t}$.
18: Send $\boldsymbol{\theta}_{n}^{t}$ to the server.
19: Store new exemplars used in the next incremental round.

feature mean of each class is the key information that helps the model preserve old knowledge. Due to the difference in label distribution, the clients owning the same class may obtain various feature means. For a specific class, when some of the feature means deviate a lot from the global distribution, the old knowledge of this class will show an increasing difference among clients' models as federated class-incremental learning continues. This can result in a disturbance within the global aggregated model (the global feature extractor in FCIDF). For this reason, we propose to share the feature means among clients and generate global feature means, so that the effect of biased feature means can be mitigated and the forgetting caused by disturbance can be relieved.

TABLE II
THE RESULTS OF DIFFERENT STORING METHODS ON CIFAR-100

	cifar_5_10_2		cifar_20_10_2	
	acc	fgt	acc	fgt
local_store	44.36	27.85	**39.36**	**25.17**
fedavg_store_w1	45.07	26.20	38.37	25.65
fedavg_store_w2	**45.33**	**24.88**	38.33	25.30

Motivated by the above idea, we first propose a simple aggregation method based on FedAvg [4] to aggregate the local feature means and generate the global feature mean of each class. We conduct several experiments to evaluate the performance of this method compared to the original storing method. There are three main steps in the method, which can directly replace the original storing step in FCIDF. First, for each class, we extract the samples' features by the current local feature extractor, and compute the feature mean. Second, the clients send the local feature means to the server, and the server calculates the global feature mean for each class using weighted average. Finally, the clients receive the global feature means of corresponding classes and store exemplars based on these feature means.

We can see that the first step and the last step are similar to the original storing method, and the key is to obtain global feature means. In FedAvg, the aggregation weights are calculated based on the number of training samples. However, we cannot upload the specific number of training samples of each class due to privacy concerns. Thus, we consider the following two weighting approaches. The first step is to calculate the feature means for each client:

$$\boldsymbol{p}_{c}^{n} = \sum_{x \in \mathcal{D}_{n,c}} \frac{1}{|\mathcal{D}_{n,c}|} f(\boldsymbol{\alpha}_{k}, \boldsymbol{\theta}_{glob}^{t}, \boldsymbol{\theta}_{k,loc}; x), \tag{5}$$

where $\boldsymbol{p}_{c}^{n}$ is the feature mean of class c on the client n, $\mathcal{D}_{n,c}$ is the local training samples of class c on the client n. Then, the first approach is to calculate the average of all feature means (w1):

$$\boldsymbol{p}_{c}^{glob} = \sum_{n \in S_{c}} \frac{1}{|S_{c}|} \cdot \boldsymbol{p}_{c}^{n}, \tag{6}$$

where $\boldsymbol{p}_{c}^{glob}$ is the global feature mean of class c, $\boldsymbol{p}_{c}^{n}$ is client n's local feature mean of class c, and S_{c} is the clients that upload class c's local feature mean. The second approach calculates the aggregation weight based on the number of clients' training samples (w2):

$$\boldsymbol{p}_{c}^{glob} = \sum_{n \in S_{c}} \frac{\mathcal{N}_{n}^{t}}{\mathcal{N}_{c}^{t}} \cdot \boldsymbol{p}_{c}^{n}, \tag{7}$$

where $\mathcal{N}_{c}^{t}$ is the total number of training samples of the clients in S_{c}. We assess the performance of both weighting approaches mentioned above and compare them to the original storing method that stores exemplars based on local feature means.

Tables II and III show the experiment results of FedAvg-based feature mean aggregation (fedavg_store_w1 and fedavg_store_w2) and local storing (local_store). We can observe that when the scale is small (cifar_5_10_2 and ilsvrc_5_10_2),

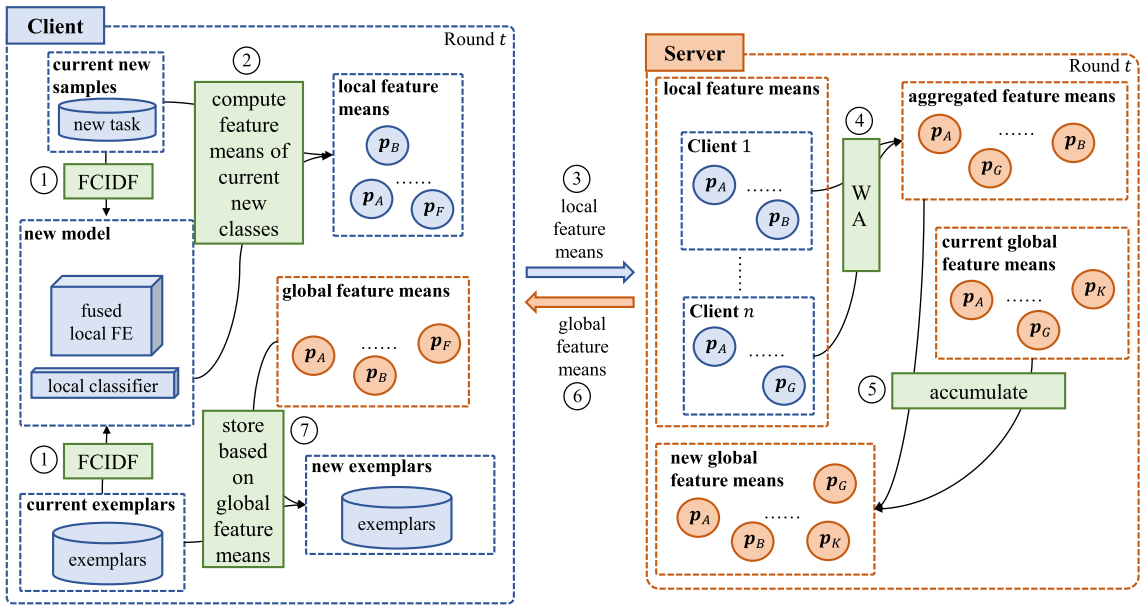

Fig. 2. Overview of AGFMS. It can be divided into the following steps: ① The client performs the local training process of FCIDF; ② The client computes the local feature means of each new class; ③ The client sends the local feature means to the server; ④ The server aggregates the local feature means using weighted average; ⑤ The server accumulates the aggregated feature means to current global feature means and obtains new global feature means; ⑥ The server sends the corresponding global feature means to the clients; ⑦ The client stores exemplars based on received global feature means.

TABLE III
THE RESULTS OF DIFFERENT STORING METHODS ON ILSVRC2012

	ilsvrc_5_10_2		ilsvrc_25_10_7	
	acc	fgt	acc	fgt
local_store	29.97	**24.49**	**38.89**	**44.98**
fedavg_store_w1	**31.92**	24.86	38.67	46.49
fedavg_store_w2	29.98	25.95	37.38	46.60

fedavg_store with both weighting approaches outperforms local_store in accuracy, with a highest improvement of 1.95%. However, when the scale becomes large (cifar_20_10_2 and ilsvrc_25_10_7), the accuracy of fedavg_store is worse than local_store. Besides, there is little improvement in forgetting under 3 evaluated scales. From these results, we can see that there is possibility of improving local performance by global feature means, but the method still possesses some disadvantages to be optimized. In federated class-incremental settings, we update the model at every round so that the feature means change over time. The unstable feature means can disturb the global feature means and mislead the exemplar selection, decreasing the reliability of global feature means. Besides, the computation and communication cost of feature means become larger over time. Thus, we should further improve the quality of global feature means to achieve better performance.

2) Accumulated Global Feature Mean Based Storing: From the experiment result of Feature Mean Aggregation based on FedAvg, we can observe that the performance is improved by feature mean aggregation in some settings. Thus, we further optimize the method and propose a novel storing method, called Accumulated Global Feature Mean based Storing (AGFMS), where the base idea is to keep the global feature mean stable. Fig. 2 shows the main process of AGFMS.

Similar to the storing method based on FedAvg, the process of AGFMS is conducted after the local training. After client n obtains the local model through FCIDF at incremental round t, we extract the features of the samples in $\mathcal{T}_n^t$ by the feature extractor $\boldsymbol{\theta}_n^t$ and compute the feature means for each new class. Then, client n sends its local feature means to the server along with the local feature extractor, and the server aggregates the local feature extractors and the feature means separately. The server first identifies global new classes and global old classes from the uploaded feature means. We denote the classes that are not observed by any client at incremental round 1 to $t-1$ as global new classes, and the rest are global old classes. At incremental round t, the server stores global feature means of all observed classes, so the classes that do not have global feature means are classified to global new classes. We denote the global feature means stored on the server currently as $\boldsymbol{p}_{glob}^{t-1}$. The server aggregates the local feature means by the weighting approaches mentioned above (w1 and w2), and the aggregated feature means are denoted as $\boldsymbol{p}_{aggr}$. For each global new class, $\boldsymbol{p}_{aggr}$ is directly used as the global feature mean. While for each global old class, we accumulate $\boldsymbol{p}_{aggr}$ to $\boldsymbol{p}_{glob}^{t-1}$ by the following:

$$\boldsymbol{p}_{glob}^{t} = (1-\gamma)\cdot \boldsymbol{p}_{glob}^{t-1} + \gamma \cdot \boldsymbol{p}_{aggr}, \tag{8}$$

where $\gamma = \exp(-\frac{t-1}{T})$ decides the ratio of current feature mean by the amount of accumulated knowledge in the global feature mean. After we obtain $\boldsymbol{p}_{glob}^{t}$, the server sends the global feature means of the corresponding classes to each client, and the clients store exemplars based on the received global feature means. Note that the clients only upload and receive the feature means of its

Algorithm 3: AGFMS.

Input: Fused feature extractor $\boldsymbol{\theta}_n^t$, new task $\mathcal{T}_n^t$, new classes $\mathcal{C}_n^t$, exemplars $\mathcal{M}_n^t$
Output: Exemplars of the next incremental round
1: Extract the features of the samples in $\mathcal{T}_n^t$ by $\boldsymbol{\theta}_n^t$.
2: Calculate the feature means $\boldsymbol{p}_n$ for each class in $\mathcal{C}_n^t$.
3: Send $\boldsymbol{p}_n$ to the server.
4: Receive the global feature means of each class in $\mathcal{C}_n^t$: $\boldsymbol{p}_{n,glob} \leftarrow \mathbf{ServerAggrMean}()$.
5: Store exemplars based on $\boldsymbol{p}_{n,glob}$ for each class in $\mathcal{C}_n^t$, and directly delete the exemplars from the end for each old class.

Algorithm 4: $\mathbf{ServerAggrMean}$.

Input: Current global feature means $\boldsymbol{p}_{glob}^{t-1}$
Output: New global feature means $\boldsymbol{p}_{glob}^{t}$
1: Receive local feature means $\boldsymbol{p}_n$ from each client. Aggregate the feature means of the same classes using (6) or (7) and obtain $\boldsymbol{p}_{aggr}$.
2: $\boldsymbol{p}_{glob}^t \leftarrow \{\}$.
3: For global new classes: $\boldsymbol{p}_{glob}^t \leftarrow \boldsymbol{p}_{glob}^t \cup \boldsymbol{p}_{aggr}$.
4: For global old classes, calculate the global feature means using (8) and update $\boldsymbol{p}_{glob}^t$.
5: Send the corresponding feature means in $\boldsymbol{p}_{glob}^t$ to the clients.

current new classes. For the old classes, we directly delete the samples from the end.

Algorithm 3 illustrates the details of AGFMS. In AGFMS, we accumulate all clients' feature means of the same classes, which makes each client benefit from the knowledge of other clients that own the same classes and compensates for the forgetting of some classes. Besides, the clients and the server exchange each class' local and global feature means only for once. This strategy can avoid the interference of the biased local feature means and keep the global feature means stable, so that each client can store the exemplars that are close to global distribution and relieve the noise within the global feature extractor.

C. Efficiency Analysis

1) Communication: In respect to dynamic feature extractor fusion, the communication cost occurs at the beginning and the end of an incremental round. At the beginning, the server only needs to send the global feature extractor $\boldsymbol{\theta}_{glob}^t$ aggregated at the last incremental round to the clients, so the communication cost remains $N \times |\boldsymbol{\theta}_{glob}^t|$ at each incremental round, where $|\cdot|$ denotes the number of parameters. In the same way, each client sends only the fused feature extractor $\boldsymbol{\theta}_n^t$ to the server after local training, and thus the cost is $N \times |\boldsymbol{\theta}_n^t|$. Note that $|\boldsymbol{\theta}_{glob}^t|$ equals to $|\boldsymbol{\theta}_n^t|$ and they remain unchanged when t increases. In respect to AGFMS, the communication cost occurs at the end of an incremental round. Each client only needs to upload and receive the global feature means of the current new classes, so it cost $2\ N \times C \times |\boldsymbol{p}|$ at each incremental round, where $|\boldsymbol{p}|$ equals to the output dimension of the feature extractor. After T incremental rounds, FCIDF requires $T \times 2\ N \times (|\boldsymbol{\theta}| + C \times |\boldsymbol{p}|)$ for communication, where $|\boldsymbol{\theta}|$ represents the number of parameters in the feature extractor.

2) Training and Memory: To perform incremental-meta learning, we need to train t meta models for R rounds separately and update $\boldsymbol{\alpha}_n^t$, $\boldsymbol{\theta}_{n,loc}^t$ and $\boldsymbol{\phi}_n^t$ of each meta model, so the training cost of incremental round t is $E \times B \times t \times R \times (|\boldsymbol{\alpha}_n^t| + |\boldsymbol{\theta}_{n,loc}^t| + |\boldsymbol{\phi}_n^t|)$, where B is the number of training batches. The training cost increases quadratically since both the number of meta models and the size of $\boldsymbol{\phi}_n^t$ grows with t.

In respect of memory, we store a fixed number of exemplars and a local model for classification on each client, so the memory cost on a single client is $M + |\boldsymbol{\theta}_n^t| + |\boldsymbol{\phi}_n^t|$, where only the $|\boldsymbol{\phi}_n^t|$ is linearly increased with t. The memory cost on server is comprised of a feature extractor and the feature means of global old classes, so it's an unfixed value.

V. Experiments

In this section, we evaluate FCIDF under different settings of the number of clients N and the number of new classes C in an incremental task. We conduct our experiments on two datasets, CIFAR-100 [37] and ILSVRC2012 [38].

A. Data Settings

CIFAR-100 contains 60,000 32×32 RGB images of 100 classes, and each class contains 500 training images and 100 testing images. *ILSVRC2012* has 1,000 classes, and each class contains 732–1,300 training images and 50 validation images. For CIFAR-100, we mix training images and testing images together, while for ILSVRC2012, we use only the training images. We randomly divide the data for training and testing under different settings.

We fix the number of incremental rounds T to 10, which means each client receives 10 incremental tasks under all scales. We group the scales by the number of new classes C and each group contains scales with various numbers of clients. There are 3 groups for both datasets, where C is set to 2, 5 and 7 respectively. The number of clients is increased to generate different scales within a group. We denote a scale as *cifar_N_T_C* and *ilsvrc_N_T_C* for CIFAR-100 and ILSVRC2012 respectively. Since the CIFAR-100 dataset does not contain enough images for large scales, we augment the dataset such that each class has 1800 images for training and testing.

To avoid the effect of data imbalance within a task, we fix the number of samples for each class within a single task to S. In other words, each incremental task is composed of $C \times S$ samples. S is set to 150 for CIFAR-100 and 250 for ILSVRC2012. We form the incremental tasks as follows. First, we shuffle and split the data of each class into several subsets of size S, and there is no overlap among all subsets. Then, each class's subsets are randomly allocated to a client sequentially until each client obtains $T \times C$ classes. Note that the subsets of the same class should be allocated to different clients, since the clients only receive new classes at each incremental round. Finally, the clients randomly divide the obtained subsets into T

TABLE IV
HYPERPARAMETER OF DIFFERENT SCALES

scale group	init_lr	num_epochs	dec_epochs
cifar_N_10_2	0.01	70	20, 30
cifar_N_10_5	0.01	70	10, 20
cifar_N_10_7	0.01	70	10, 20
ilsvrc_N_10_2	0.01	40	10, 20
ilsvrc_N_10_5 (N = 5,10,20)	0.01	40	10, 20
ilsvrc_N_10_5 (N = 15,25)	0.001	40	–
ilsvrc_N_10_7	0.01	40	10, 20

Init_lr is the Initial Learning Rate, num_epochs is the Number of Epochs, dec_epochs is the Epochs to Divide the Learning Rate by 10.

incremental tasks, each with C classes. The incremental tasks are further divided into training set and testing set by the ratio of 8:2.

B. Implementation Details

1) Configurations: We adopt the LeNet [39] model for training, where the feature extractor consists of two convolutional layers and two fully-connected layers, and the output layer is the classifier. The number of exemplars stored on each client is calculated by $T \times C \times 12$. We set the batch size to 100 for all scales and other hyperparameters are concluded in Table IV.

2) Metrics: We calculate the top-1 accuracy for ilsvrc_N_10_2 and all scales of CIFAR-100, and top-5 accuracy for other scales of ILSVRC2012. The accuracy of each incremental round is averaged among clients, and the final accuracy is calculated by the mean of all incremental rounds. Note that the first round is not included when calculating the final accuracy since it does not perform class-incremental learning.

We also evaluate the forgetting of our method, which can reflect the model's performance on old classes. For incremental round $t > 1$, the forgetting is calculated by the following:

$$fgt_t = \frac{1}{C_{old}} \sum_{i=1}^{C_{old}} \max_{j \in 1,\ldots,t-1} acc_{i,j} - acc_{i,t}, \tag{9}$$

where C_{old} is the number of old classes, $acc_{i,j}$ is the accuracy of class i at incremental round j.

3) Baselines: We divide the compared methods into two categories based on the class-incremental learning methods: distillation-based methods (*distIL*) and meta-learning based methods (*metaIL*). The distillation-based methods add a knowledge distillation term to the loss function and perform batch gradient descent, while the meta-learning based methods perform incremental-meta learning as described in this paper. We compare FCIDF to two federated learning method: *Replace* and *Fedprox* [28]. *Replace* is the traditional FedAvg [4] method that directly replaces the local model with the aggregated global model, and *Fedprox* utilizes a proximal term to keep the models close to the global model. Single class-incremental learning (*Local*) is also performed to observe the effect of our federated learning method. We combine the above class-incremental learning methods with federated learning methods and generate 5 baselines: *distIL_Replace*, *metaIL_Replace*, *metaIL_Local*, *metaIL_Fedprox*, *distIL_FCIDF*.

Moreover, we also compare our method with the state-of-the-art federated class-incremental learning methods, namely *GLFC* [16], which utilizes the gradients of samples to help the server choose the best old model for distillation. To ensure a fair comparison, we report the average accuracy and forgetting of all local models after all incremental rounds.

C. Experimental Results

1) Overall Performance: We report the average accuracy and forgetting of all incremental rounds, and we also rank the methods from 1 to 6 by accuracy on each scale to better compare their overall performance.

Tables V–VII shows the average accuracy and forgetting on CIFAR-100. We can observe that FCIDF performs the best in accuracy when meta-learning is used (metaIL_FCIDF), and the distillation loss degrades the performance of FCIDF (distIL_FCIDF). This is because the learning of new knowledge is constrained by knowledge distillation and thus the resulting model performs poorly in new classes. metaIL_FCIDF obtains lower forgetting than metaIL_Replace, which proves our motivation to relieve the catastrophic forgetting of replacing strategy. metaIL_Local underperforms metaIL_FCIDF in both accuracy and forgetting, and thus shows the advantages of FCIDF's collaborating strategy, which means metaIL_FCIDF performs positive knowledge transfer among clients. Even though metaIL_Fedprox achieves the best forgetting on all scales, it still performs worse than metaIL_FCIDF in accuracy.

Figs. 5 and 6 illustrates the accuracy and forgetting curves of meta-learning based methods. We can observe that in the later rounds, the accuracy of metaIL_FCIDF has a certain improvement compared with other methods, while the forgetting is only worse than metaIL_Fedprox where the model update is constrained to preserve old knowledge. Moreover, the forgetting curve of metaIL_FCIDF draws an increasing difference from that of metaIL_Replace and metaIL_Local, which indicates our method achieves positive knowledge transfer among clients. The accuracy of metaIL_Fedprox drops rapidly in the early stage but gets stable later, even if it learns little new knowledge. We can observe from the forgetting curves that metaIL_Fedprox is the best to preserve old knowledge, and thus the accuracy of old classes compensates for the rapid drop in new classes' accuracy. The curves of metaIL_Replace and metaIL_Local are close both in accuracy and forgetting, which means the replacing strategy might not take a significant effect on transferring global knowledge.

2) Impact of the Number of Clients: As shown in Tables V–X, where each table represents the results of a scale group, we increase the number of clients within a scale group. In respect of accuracy, we can see that metaIL_FCIDF tends to rank higher as the number of clients increases, which proves its generalization capability over a larger network. Besides, the ranking of metaIL_FCIDF does not decrease significantly over different numbers of clients. In contrast, the ranking of other methods changes exceedingly when the number of clients changes, like metaIL_Fedprox in Table IX, where the ranking drops from 1 to 6. Some methods remain a low ranking over different numbers

TABLE V
THE ACCURACY AND FORGETTING ON CIFAR-100 AND $C = 2$

	cifar_5_10_2		**cifar_10_10_2**		**cifar_15_10_2**		**cifar_20_10_2**	
	acc	fgt	acc	fgt	acc	fgt	acc	fgt
distIL_Replace	46.16	34.67	38.12	33.72	36.50	34.15	37.59	34.32
metaIL_Replace	47.23	32.31	38.35	31.99	37.76	31.50	39.46	32.54
metaIL_Local	46.25	32.78	38.29	31.99	36.76	33.01	39.45	32.65
metaIL_Fedprox	46.19	21.47	37.57	**19.42**	36.95	**19.24**	37.24	20.34
distIL_FCIDF	44.07	35.65	37.03	34.40	36.54	34.19	38.26	35.36
GLFC	45.38	**19.34**	37.15	20.58	37.56	24.73	38.99	**19.60**
metaIL_FCIDF (ours)	**47.30**	25.08	**38.56**	24.09	**38.30**	23.75	**40.24**	23.86

TABLE VI
THE ACCURACY AND FORGETTING ON CIFAR-100 AND $C = 5$

	cifar_5_10_5		**cifar_10_10_5**		**cifar_15_10_5**		**cifar_20_10_5**	
	acc	**fgt**	**acc**	**fgt**	**acc**	**fgt**	**acc**	**fgt**
distIL_Replace	29.54	27.20	24.12	25.21	23.54	25.32	23.65	25.29
metaIL_Replace	31.08	23.79	26.28	20.73	26.08	21.31	26.37	20.97
metaIL_Local	31.06	23.66	26.22	22.21	25.98	21.90	26.52	22.52
metaIL_Fedprox	28.37	**12.85**	25.51	**10.85**	23.77	**12.33**	24.60	**12.31**
distIL_FCIDF	30.33	26.81	25.40	24.80	24.97	25.03	26.56	25.23
GLFC	**32.97**	19.48	**27.94**	14.35	**27.43**	15.70	26.59	21.02
metaIL_FCIDF (ours)	31.83	18.39	27.13	16.84	26.30	16.56	**27.40**	16.55

TABLE VII
THE ACCURACY AND FORGETTING ON CIFAR-100 AND $C = 7$

	cifar_5_10_7		**cifar_8_10_7**		**cifar_10_10_7**		**cifar_15_10_7**	
	acc	**fgt**	**acc**	**fgt**	**acc**	**fgt**	**acc**	**fgt**
distIL_Replace	26.60	25.25	22.49	23.81	20.62	22.99	19.92	22.20
metaIL_Replace	29.00	22.67	25.38	20.96	24.83	20.31	21.74	19.03
metaIL_Local	28.48	22.70	25.08	21.67	23.64	20.99	21.62	19.68
metaIL_Fedprox	26.91	**11.56**	23.57	**12.30**	22.53	**12.23**	22.10	**10.84**
distIL_FCIDF	27.58	24.44	25.58	24.28	24.01	23.89	22.47	23.15
GLFC	28.21	20.56	25.56	21.76	24.93	20.70	**23.39**	18.91
metaIL_FCIDF (ours)	**29.38**	18.56	**27.03**	17.66	**25.59**	16.14	23.26	16.04

TABLE VIII
THE ACCURACY AND FORGETTING ON ILSVRC2012 AND $C = 2$

	ilsvrc_5_10_2		**ilsvrc_10_10_2**		**ilsvrc_15_10_2**		**ilsvrc_20_10_2**		**ilsvrc_25_10_2**	
	acc	**fgt**	**acc**	**fgt**	**acc**	**fgt**	**acc**	**fgt**	**acc**	**fgt**
distIL_Replace	28.96	44.15	28.83	42.45	28.55	42.26	28.15	42.91	28.10	41.88
metaIL_Replace	31.89	31.51	32.22	31.84	**32.09**	30.32	**31.18**	29.71	**31.25**	29.91
metaIL_Local	**32.08**	30.58	32.27	31.81	30.93	32.20	30.19	32.23	28.33	31.81
metaIL_Fedprox	30.18	**19.08**	28.69	**20.58**	29.50	**20.31**	28.91	**19.70**	27.08	**20.13**
distIL_FCIDF	27.89	50.64	29.66	47.30	28.41	49.39	27.54	48.62	27.20	48.14
GLFC	30.29	25.96	31.73	20.84	30.60	21.60	29.25	24.44	26.26	21.14
metaIL_FCIDF (ours)	30.74	23.34	**32.45**	22.04	31.60	23.65	31.07	22.00	30.77	21.82

TABLE IX
THE ACCURACY AND FORGETTING ON ILSVRC2012 AND $C = 5$

	ilsvrc_5_10_5		**ilsvrc_10_10_5**		**ilsvrc_15_10_5**		**ilsvrc_20_10_5**		**ilsvrc_25_10_5**	
	acc	**fgt**	**acc**	**fgt**	**acc**	**fgt**	**acc**	**fgt**	**acc**	**fgt**
distIL_Replace	44.12	53.21	42.85	52.16	48.32	52.52	43.33	51.33	47.25	53.92
metaIL_Replace	47.73	55.82	46.87	54.00	**51.01**	40.27	47.59	52.89	47.52	43.48
metaIL_Local	45.82	59.46	44.59	58.92	49.00	44.94	45.46	58.87	47.75	45.53
metaIL_Fedprox	48.02	**33.04**	**49.50**	**29.88**	48.37	**24.90**	48.50	**30.10**	45.69	**26.79**
distIL_FCIDF	47.55	56.99	47.28	55.61	46.98	56.39	47.08	55.77	46.26	56.78
GLFC	47.88	41.86	48.48	45.85	48.04	46.75	**49.71**	42.26	48.68	40.37
metaIL_FCIDF (ours)	**50.05**	46.43	49.21	45.09	50.62	40.42	49.26	45.38	**49.12**	40.56

TABLE X
THE ACCURACY AND FORGETTING ON ILSVRC2012 AND $C = 7$

	ilsvrc_5_10_7		ilsvrc_10_10_7		ilsvrc_15_10_7		ilsvrc_20_10_7		ilsvrc_25_10_7	
	acc	fgt	acc	fgt	acc	fgt	acc	fgt	acc	fgt
distIL_Replace	35.14	47.90	34.26	46.84	34.37	46.56	33.35	46.37	32.96	45.85
metaIL_Replace	39.53	52.15	39.06	51.07	39.00	51.32	38.43	51.22	37.62	51.34
metaIL_Local	38.67	54.89	37.66	55.54	37.29	56.62	36.53	56.54	36.33	55.68
metaIL_Fedprox	**41.53**	**27.30**	**42.69**	**26.45**	41.22	**26.38**	**40.89**	**26.12**	**41.69**	**26.69**
distIL_FCIDF	40.11	53.39	39.08	53.55	38.43	54.03	36.94	54.35	37.22	53.98
GLFC	40.08	37.26	42.64	44.43	**41.36**	45.83	38.15	47.20	39.30	46.01
metaIL_FCIDF (ours)	41.43	43.92	41.14	44.08	40.95	45.03	39.53	46.18	40.65	43.22

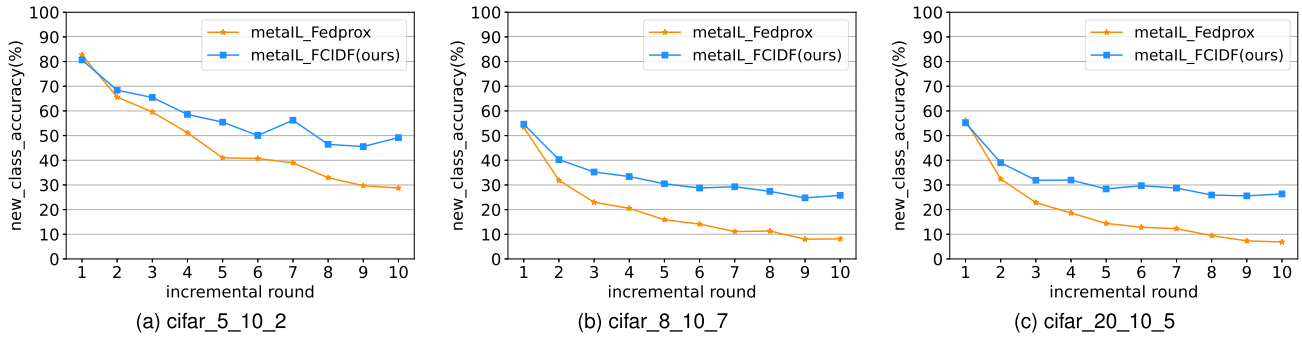

Fig. 3. Average accuracy of new classes' data on 3 scales of CIFAR-100. metaIL_Fedprox declines tremendously compared to metaIL_FCIDF, and a difference of up to 20% is observed at the last incremental round.

TABLE XI
AVERAGE RANKING OF ACCURACY ON ILSVRC2012

method \ scale group	N_10_2	N_10_5	N_10_7	overall
distIL_Replace	5.6	6.2	7.0	6.27
metaIL_Replace	**1.6**	3.6	4.2	3.13
metaIL_Local	2.4	4.8	6.0	4.40
metaIL_Fedprox	5.6	3.4	**1.2**	3.40
distIL_FCIDF	6.2	5.4	4.4	5.33
GLFC	4.6	3.0	2.8	3.47
metaIL_FCIDF (ours)	2.0	**1.6**	2.4	**2.00**

of clients in several scale groups, like distIL_FCIDF in Tables V and IX, metaIL_Local in Table X. In respect of forgetting, metaIL_Fedprox ranks the highest over different numbers of clients, and metaIL_FCIDF follows behind, which is not affected by the number of clients.

The reason is that Fedprox does not want the model to change a lot from the global model, which helps to preserve the old knowledge well but fails to learn new knowledge. As illustrated in Fig. 3, the accuracy of new classes' data of metaIL_Fedprox declines rapidly as the new tasks are trained continually, while that of metaIL_FCIDF is more stable.

Tables VIII–X shows the experimental results on ILSVRC2012. It can be observed that the accuracy ranking varies over different scales of ILSVRC2012. To compare the overall performance in accuracy more explicitly, we compute the average ranking of the compared methods on each scale group, as shown in Table XI, and the overall column reports the mean ranking of all evaluated scales. Though metaIL_Replace and metaIL_Fedprox have the highest rank on ilsvrc_N_10_2 and ilsvrc_N_10_7 respectively, their rankings decrease exceedingly on the other scale groups. metaIL_FCIDF remains a stable ranking on different scales, and thus obtains the highest overall ranking of accuracy on ILSVRC2012. The forgetting of metaIL_FCIDF is only lower than that of metaIL_Fedprox, which is similar to CIFAR-100. We also compare their new classes' accuracy on ILSVRC2012, as shown in Fig. 4, and observe that the new classes' accuracy of metaIL_Fedprox drops tremendously compared to metaIL_FCIDF, leading to its poor performance in overall accuracy. Overall, we can draw the same conclusion as CIFAR-100 on ILSVRC2012.

3) Impact of the Number of New Classes: We report the average ranking of accuracy on ILSVRC2012 in Table XI. From Table XI, we can observe that when the number of new classes changes from 2 to 7, the average ranking of metaIL_Replace declines, drawing an increasing difference from metaIL_FCIDF. This indicates that metaIL_FCIDF's ability to alleviate the forgetting of replacing strategy gets more useful as the number of new classes increases. metaIL_Local's ranking also declines as the number of new classes increases and is lower than metaIL_FCIDF, revealing the advantage of global knowledge transfer when learning more classes. Though metaIL_Fedprox remains a low ranking on CIFAR-100, it obtains a higher ranking when the number of new classes increases on ILSVRC2012. This is because the number of old classes gets much more than that of new classes as the incremental learning proceeds, and the method that can preserve more old knowledge, i.e. metaIL_Fedprox, prevails in this situation.

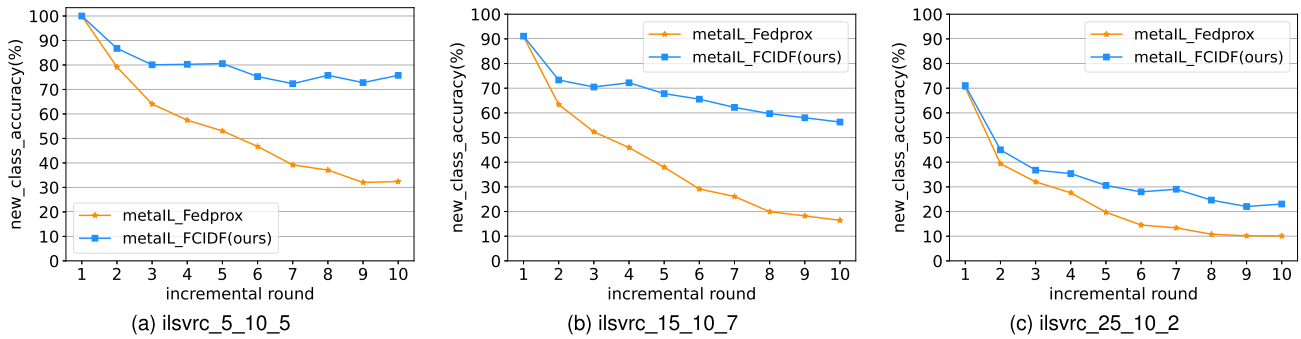

Fig. 4. Average accuracy of new classes' data on 3 scales of ILSVRC2012. metaIL_Fedprox declines tremendously compared to metaIL_FCIDF, and a difference of about 40% is reached at the last incremental round when the number of new classes is 5 and 7.

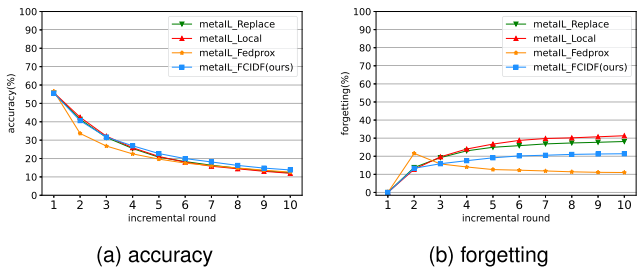

Fig. 5. Accuracy and forgetting on cifar_20_10_5.

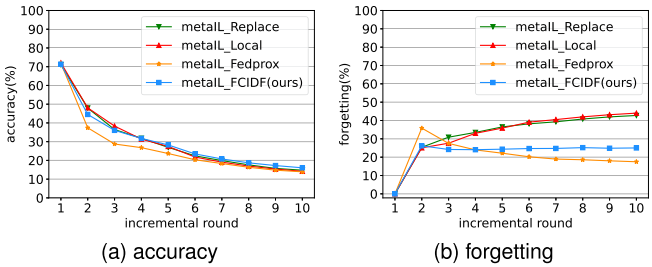

Fig. 6. Accuracy and forgetting on ilsvrc_10_10_2.

In conclusion, metaIL_FCIDF can be further optimized in a way that constrains the updating direction to keep a balance between preserving old knowledge and learning new knowledge.

4) Impact of Classifier Aggregation: We further study the impact of aggregating heterogeneous classifiers. The aggregation method we used for heterogeneous classifiers is derived from FedAvg, where we aggregate the classifiers' parameters of the same class by weighted average and obtain the global classifier parameters of each class. The global classifier parameters will replace the corresponding part of the local classifiers and be trained locally.

Tables XII and XIII shows the performance of classifier aggregation on CIFAR-100 and ILSVRC2012, where *fully_aggr* aggregates classifier and *fe_aggr* doesn't. We can observe from the results of CIFAR-100 that the accuracy declines and the forgetting rises when aggregating classifier in the most settings, especially in the cases that the number of new classes increases. In the other settings, the effect of classifier aggregation is not obvious compared to its computation and communication cost, which degrades the necessity of aggregating classifier. On ILSVRC2012, we can draw the same conclusion as CIFAR-100. Although the forgetting is decreased by classifier aggregation when the number of classes increases, the accuracy is still close to the case that doesn't aggregate classifier and even drops. The experiment results prove that the traditional aggregation method performs poorly on heterogeneous classifiers under our method. The data distributions of the clients owning the same class may be different, so that the class relationships within the classifiers varies even if the clients have the same class. Aggregating heterogeneous classifiers by FedAvg can critically disturb the predictive bounds of the clients, and thus fail to transfer the global knowledge within the classifiers.

5) Evaluation of AGFMS: In this section, we evaluate the effect of global feature means on local model performance. We compare AGFMS to the original local storing method which stores exemplars based on local feature means under FCIDF settings. We choose a small scale and a large scale from each scale group and conduct the experiments. Tables XIV and XVII show the average accuracy and forgetting of two AGFMS settings and local storing.

From Tables XIV and XV, we can observe that AGFMS obtains higher overall performance under all scales of CIFAR-100, and the performances of AGFMS using different weighting approaches are similar. When the scale is relatively small (Table XIV), both AGFMS settings outperform local_store on accuracy under most scales, where AGFMS_w1 achieves up to 2.09% improvement and AGFMS_w2 3.88% compared to local_store. In respect to forgetting, AGFMS_w1 obtains a decline of up to 2.71% and AGFMS_w2 3.51% respectively. When the scale is large (Table XV), AGFMS obtains less improvement on accuracy and forgetting, and the highest improvement of accuracy is 0.65% achieved by AGFMS_w1 on cifar_20_10_2.

The experiment results on ILSVRC2012 are shown in Tables XVI and XVII. In respect to accuracy of small scales (Table XVI), AGFMS_w1 always outperforms local_store, and AGFMS achieves an improvement of up to 1.50% on ilsvrc_5_10_2 compared to local storing. When the scale is relatively large, AGFMS obtains little increment in accuracy, where the highest increment achieved on ilsvrc_15_10_2 is 0.96%.

TABLE XII
IMPACT OF AGGREGATING CLASSIFIER ON CIFAR-100

	cifar_5_10_2		cifar_10_10_2		cifar_15_10_2		cifar_20_10_2	
	acc	fgt	acc	fgt	acc	fgt	acc	fgt
metaIL_FCIDF fully_aggr	46.07	25.95	39.42	23.44	38.68	25.26	39.07	25.95
metaIL_FCIDF fe_aggr (ours)	47.30	25.08	38.56	24.09	38.30	23.75	40.24	23.86
	cifar_5_10_5		**cifar_10_10_5**		**cifar_15_10_5**		**cifar_20_10_5**	
	acc	fgt	acc	fgt	acc	fgt	acc	fgt
metaIL_FCIDF fully_aggr	32.75	19.69	25.41	18.38	26.16	18.18	26.91	17.50
metaIL_FCIDF fe_aggr (ours)	31.83	18.39	27.13	16.84	26.30	16.56	27.40	16.55
	cifar_5_10_7		**cifar_8_10_7**		**cifar_10_10_7**		**cifar_15_10_7**	
	acc	fgt	acc	fgt	acc	fgt	acc	fgt
metaIL_FCIDF fully_aggr	29.80	18.86	25.56	18.81	23.89	17.75	18.64	17.59
metaIL_FCIDF fe_aggr (ours)	29.38	18.56	27.03	17.66	25.59	16.14	23.26	16.04

TABLE XIII
IMPACT OF AGGREGATING CLASSIFIER ON ILSVRC2012

	ilsvrc_5_10_2		ilsvrc_10_10_2		ilsvrc_15_10_2		ilsvrc_20_10_2		ilsvrc_25_10_2	
	acc	fgt	acc	fgt	acc	fgt	acc	fgt	acc	fgt
metaIL_FCIDF fully_aggr	32.49	25.95	31.39	25.17	31.44	25.57	31.39	24.02	30.96	24.41
metaIL_FCIDF fe_aggr (ours)	30.74	23.34	32.45	22.04	31.60	23.65	31.07	22.00	30.77	21.82
	ilsvrc_5_10_5		**ilsvrc_10_10_5**		**ilsvrc_15_10_5**		**ilsvrc_20_10_5**		**ilsvrc_25_10_5**	
	acc	fgt	acc	fgt	acc	fgt	acc	fgt	acc	fgt
metaIL_FCIDF fully_aggr	51.29	43.96	48.49	43.11	50.03	40.00	49.13	44.09	48.62	41.03
metaIL_FCIDF fe_aggr (ours)	50.05	46.43	49.21	45.09	50.62	40.42	49.26	45.38	49.12	40.56
	ilsvrc_5_10_7		**ilsvrc_10_10_7**		**ilsvrc_15_10_7**		**ilsvrc_20_10_7**		**ilsvrc_25_10_7**	
	acc	fgt	acc	fgt	acc	fgt	acc	fgt	acc	fgt
metaIL_FCIDF fully_aggr	40.93	41.83	41.01	42.39	40.75	42.66	39.92	42.99	38.23	42.93
metaIL_FCIDF fe_aggr (ours)	41.43	43.92	41.14	44.08	40.95	45.03	39.53	46.18	40.65	43.22

TABLE XIV
COMPARISON OF AGFMS AND LOCAL STORING ON CIFAR-100 (SMALL SCALE)

	cifar_5_10_2		cifar_10_10_5		cifar_8_10_7	
	acc	fgt	acc	fgt	acc	fgt
local_storing	44.36	27.85	25.79	17.27	25.13	**18.03**
AGFMS_w1	46.45	25.14	**26.50**	16.52	**25.60**	18.17
AGFMS_w2	**48.24**	**24.34**	26.02	**16.51**	25.12	18.79

TABLE XV
COMPARISON OF AGFMS AND LOCAL STORING ON CIFAR-100 (LARGE SCALE)

	cifar_20_10_2		cifar_20_10_5		cifar_15_10_7	
	acc	fgt	acc	fgt	acc	fgt
local_storing	39.36	25.17	**27.14**	**16.37**	23.10	**16.78**
AGFMS_w1	**40.01**	**24.74**	26.12	16.73	22.40	16.88
AGFMS_w2	38.75	25.13	26.60	17.16	**23.41**	17.02

TABLE XVI
COMPARISON OF AGFMS AND LOCAL STORING ON ILSVRC2012 (SMALL SCALE)

	ilsvrc_5_10_2		ilsvrc_10_10_5		ilsvrc_5_10_7	
	acc	fgt	acc	fgt	acc	fgt
local_storing	29.97	24.49	48.22	46.22	40.82	45.26
AGFMS_w1	31.00	24.04	**48.89**	**45.14**	40.95	43.39
AGFMS_w2	**31.47**	**23.17**	46.27	47.17	**41.71**	**42.36**

TABLE XVII
COMPARISON OF AGFMS AND LOCAL STORING ON ILSVRC2012 (LARGE SCALE)

	ilsvrc_15_10_2		ilsvrc_20_10_5		ilsvrc_25_10_7	
	acc	fgt	acc	fgt	acc	fgt
local_storing	30.53	23.65	48.25	47.26	**38.89**	44.98
AGFMS_w1	**31.49**	**22.77**	48.32	46.82	38.01	45.57
AGFMS_w2	30.93	22.89	**48.70**	**46.29**	38.73	**44.75**

In respect to the forgetting, AGFMS outperforms local storing on most settings, which proves its effectiveness on choosing better exemplars. A improvement of up to 2.90% is obtained on ilsvrc_5_10_7.

In conclusion, both the two weighting approaches of AGFMS obtain better overall performance than local storing on two evaluated datasets. AGFMS improves the accuracy and forgetting on most scales, where the improvement is up to 3.88% on accuracy and 3.51% on forgetting. These results indicate that the accumulated global feature means can help clients learn its local data better, and compensate for the forgetting of old classes.

VI. CONCLUSION

We studied the problem of federated class-incremental learning, where each client collaborates with other clients and learns continuously from new classes' data. We aimed to learn an incremental and personalized model that can relieve catastrophic

forgetting and non-IID data simultaneously for each client. To achieve this goal, we proposed FCIDF, which fuses the global feature extractor with the local feature extractor via a dynamic rate and thus extracts the relevant global knowledge for each client. Furthermore, we leveraged meta-learning to perform the local class-incremental learning, which can better involve both the old and new knowledge in the personalized training. Experimental results showed that our method outperforms the baseline methods on a wide range of scales. We also proved that aggregating heterogeneous classifier in FCIDF is unnecessary through several experiments. To further leverage the global information, we proposed a novel storing method called AGFMS, where the local feature means are accumulated to generate global feature means, and the clients store exemplars based on these global feature means. AGFMS can help the clients store better exemplars and compensate for the local forgetting, since the local feature means of the same classes are shared among clients. We can observe the effectiveness of AGFMS from the experiment results. Our future work is to consider how to decrease the cost when the model structure becomes complicated, and we also find the possibility of generalizing AGFMS to other SOTA storing methods.

REFERENCES

[1] V. Murugan, V. R. Vijaykumar, and A. Nidhila, "A deep learning RCNN approach for vehicle recognition in traffic surveillance system," in *Proc. IEEE Int. Conf. Commun. Signal Process.*, 2019, pp. 157–160.

[2] M. Chen et al., "Smart home 2.0: Innovative smart home system powered by botanical IoT and emotion detection," *Mobile Netw. Appl.*, vol. 22, no. 6, pp. 1159–1169, 2017.

[3] Z. Zhu, G. Han, G. Jia, and L. Shu, "Modified densenet for automatic fabric defect detection with edge computing for minimizing latency," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9623–9636, Oct. 2020.

[4] B. McMahan et al., "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Stat.*, 2017, pp. 1273–1282.

[5] H. Wang et al., "Federated learning with matched averaging," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–13.

[6] Y. Zhao et al., "Federated learning with non-IID data," 2018, *arXiv: 1806.00582*.

[7] M. Duan et al., "Astraea: Self-balancing federated learning for improving classification accuracy of mobile deep learning applications," in *Proc. IEEE Int. Conf. Comput. Des.*, 2019, pp. 246–254.

[8] W. Hao et al., "Towards fair federated learning with zero-shot data augmentation," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. Workshops*, 2021, pp. 3310–3319.

[9] W. Zhang et al., "Client selection for federated learning with non-IID data in mobile edge computing," *IEEE Access*, vol. 9, pp. 24462–24474, 2021.

[10] C. Briggs, Z. Fan, and P. Andras, "Federated learning with hierarchical clustering of local updates to improve training on non-IID data," in *Proc Int. Joint Conf. Neural Netw.*, 2020, pp. 1–9.

[11] A. Ghosh, J. Chung, D. Yin, and K. Ramchandran, "An efficient framework for clustered federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 19586–19597.

[12] X. Tang, S. Guo, and J. Guo, "Personalized federated learning with contextualized generalization," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 2241–2247.

[13] J. Zhang et al., "Parameterized knowledge transfer for personalized federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 10092–10104.

[14] B. Sun, H. Huo, Y. Yang, and B. Bai, "PartialFed: Cross-domain personalized federated learning via partial initialization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 23309–23320.

[15] Y. Ma et al., "Continual federated learning based on knowledge distillation," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 2182–2188.

[16] J. Dong et al., "Federated class-incremental learning," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10154–10163.

[17] J. Yoon et al., "Federated continual learning with weighted inter-client transfer," in *Proc. Mach. Learn. Res.*, 2021, pp. 12073–12086.

[18] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2935–2947, Dec. 2018.

[19] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit*, 2017, pp. 5533–5542.

[20] F. M. Castro et al., "End-to-end incremental learning," in *Proc. 15th Eur. Conf. Comput. Vis.*, 2018, pp. 241–257.

[21] Y. Wu et al., "Large scale incremental learning," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 374–382.

[22] J. Dong et al., "No one left behind: Real-world federated class-incremental learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 4, pp. 2054–2070, Apr. 2024.

[23] H. Guan, Y. Wang, X. Ma, and Y. Li, "DCIGAN: A distributed class-incremental learning method based on generative adversarial networks," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Soc. Comput. Netw.*, 2019, pp. 768–775.

[24] S. Babakniya et al., "A data-free approach to mitigate catastrophic forgetting in federated class incremental learning for vision tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 66408–66425.

[25] J. Zhang, C. Chen, W. Zhuang, and L. Lyu, "TARGET: Federated class-continual learning via exemplar-free distillation," in *Proc IEEE Int. Conf. Comput. Vis.*, 2023, pp. 4759–4770.

[26] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *Proc. IEEE Int. Conf. Commun.*, 2019, pp. 1–7.

[27] J. Park, D.-J. Han, M. Choi, and J. Moon, "Sageflow: Robust federated learning against both stragglers and adversaries," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 840–851.

[28] T. Li et al., "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, 2020, pp. 429–450.

[29] S. P. Karimireddy et al., "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Machin. Learn.*, 2020, pp. 5132–5143.

[30] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017.

[31] G. Hinton et al., "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.

[32] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1126–1135.

[33] A. Nichol, J. Achiam, and J. Schulman, "On first-order meta-learning algorithms," 2018, *arXiv: 1803.02999*.

[34] Q. Liu et al., "Incremental few-shot meta-learning via indirect discriminant alignment," in *Proc. 16th Eur. Conf. Comput. Vis.*, 2020, pp. 685–701.

[35] J. Rajasegaran et al., "iTAML: An incremental task-agnostic meta-learning approach," in *Proc IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 13585–13594.

[36] C. Liu, X. Qu, J. Wang, and J. Xiao, "FedET: A communication-efficient federated class-incremental learning framework based on enhanced transformer," in *Proc. Int. Joint Conf. Artif. Intell.*, 2023, pp. 3984–3992.

[37] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep. TR-2009, 2009.

[38] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.

[39] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

Yanyan Lu received the BE degree in software engineering from the South China University of Technology, Guangzhou, China, in 2020. She is currently working toward the master's degree with the School of Software Engineering, South China University of Technology. Her research interests lie in distributed machine learning, edge computing, and intelligence.

Lei Yang (Member, IEEE) received the BS degree in electronic engineering from Wuhan University, in 2007, the MS degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences, in 2010, and the PhD degree in computer science from The Hong Kong Polytechnic University, in May 2014. He is currently a professor with the School of Software Engineering, South China University of Technology (SCUT). Before he joined SCUT in, 2016, he has been working as a postdoctoral fellow with the Department of Computing, The Hong Kong Polytechnic University. His research interest is cloud and edge computing, distributed machine learning and federated learning. He has published more than 50 papers in major international journals and conference proceedings including top journals like *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Cloud Computing*, *ACM Transactions on Knowledge Discovery from Data* and top conferences like IEEE INFOCOM and IEEE PERCOM. He also directed several research and development projects from both government and industry agencies.

Hao-Rui Chen received the BE and ME degrees from the South China University of Technology, China, in 2020 and 2023, respectively. He is currently working toward the doctor degree with the School of Software Engineering, South China University of Technology. His research interests include edge computing and edge machine learning.

Jiannong Cao (Fellow, IEEE) received the BSc degree in computer science from Nanjing University, Nanjing, China, in 1982, and the MSc and PhD degrees in computer science from Washington State University, Pullman, WA, USA, in 1986 and 1990, respectively. From 2011 to 2017, he served the department head. He is currently the Otto Poon Charitable Foundation professor in data science and the chair professor of distributed and mobile computing with the Department of Computing, The Hong Kong Polytechnic University (PolyU), Hong Kong. He is also the dean of the Graduate School, the director of the Research Institute for Artificial Intelligence of Things, PolyU, and the director of the Internet and Mobile Computing Laboratory. He was the founding director and is currently the associate director of PolyU's University Research Facility in Big Data Analytics. His research interests include distributed systems and blockchain, wireless sensing and networking, Big Data and machine learning, and mobile cloud and edge computing. He is a member of Academia Europaea, a fellow of the China Computer Federation (CCF), and an ACM distinguished member.

Wanyu Lin (Member, IEEE) received the BEngr degree from the School of Electronic Information and Communications, Huazhong University of Science and Technology, China, the MPhil degree from the Department of Computing, The Hong Kong Polytechnic University, and the PhD degree from the Department of Electrical and Computer Engineering, University of Toronto. Her research interests include graph machine learning, trustworthy machine learning, data privacy, and model interpretability. She has served as associate editor for *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)*.

Saiqin Long received the PhD degree in computer applications technology from the South China University of Technology, Guangzhou, China, in 2014. She is currently a professor with the College of Information Science and Technology, Jinan University, China. Her research interests include cloud computing, edge computing, parallel and distributed systems, and Internet of Things. She has published 20+ refereed papers in these areas, most of which are published in premium conferences and journals, including *IEEE Transactions on Services Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Mobile Computing*, etc. She is a member of Chinese Computer Federation (CCF).